

Evaluating an infix expression (like  $3 + 5 * 2$ ) requires two stacks: a **value stack** for numbers and an **operator stack** for signs (+, -, \*, /). This method handles operator order and brackets safely from left to right.

### The Two Stacks

- **Value Stack:** Holds numbers or resolved answers.
- **Operator Stack:** Holds math signs and open brackets (.

### Step-by-Step Rules

- Scan the string from **left to right** to read each item.
- **If you see a number:** Push it straight onto the **value stack**.
- **If you see an open bracket (:** Push it onto the **operator stack**.
- **If you see a math sign (+, -, \*, /):**
  - Check the top sign on the **operator stack**.
  - If that top sign has **higher or equal priority** than the new sign, pop it. Also pop two numbers from the **value stack**. Do the math. Push the answer back to the **value stack**.
  - Repeat this until the top stack sign has lower priority or is an open bracket (.
  - Push your new sign onto the **operator stack**.
- **If you see a close bracket ):**
  - Pop signs and numbers and do the math until you find the matching open bracket (. Remove the open bracket (.
- **Finish up:**
  - When you reach the end of the expression string, pop remaining signs and numbers one by one. Do the math and push results until the **operator stack** is empty.
  - The final single number left in the **value stack** is your total answer.